

→ linked list

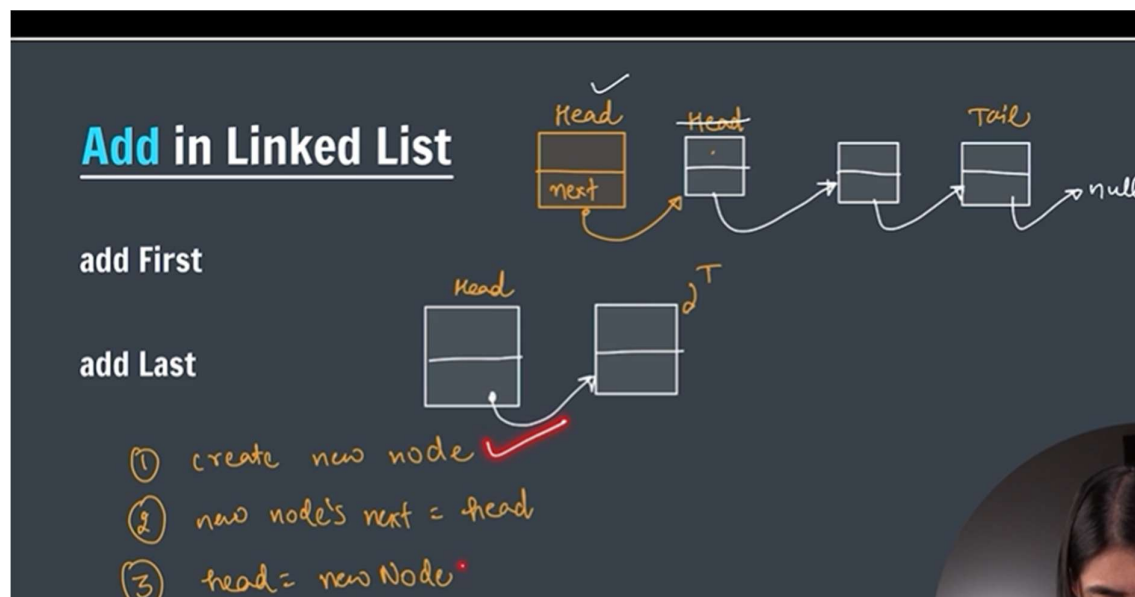
-> in linked list there are nodes in nodes two parts in that data, next

In data it stores the type of the variable

In next it stores the address of the next variable



→ we create some methods to add the nodes in first or last



→ addLast

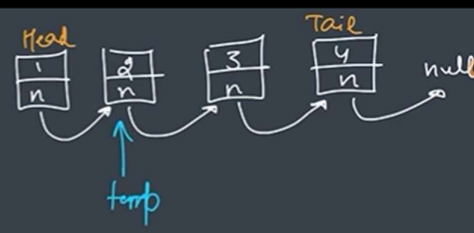
→ print a linkedlist

Print a Linked List

temp → 1 → 2 → 3 → 4 → null
Head Tail

```
Node temp = head;  
while (temp != null) {  
    print(temp.data)  
    temp = temp.next  
}
```

1



→ add in the middle— $O(n)$

Add in the Middle

$O(n)$

add(index, data)

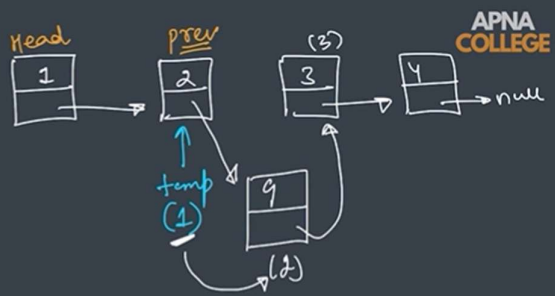
idx = 2
data = 9 ✓

```
Node temp = head;  
i = 0;  
while (i < idx) {  
    temp = temp.next;  
    i++;  
}
```

temp (prev)

- 1) newNode.next = temp.next
- 2) temp.next = newNode

temp (1)



→ size of an linkedlist

→ remove in a linkedlist

Remove first

Remove last

→ iterative search

→ recursive search

→ reverse a linkedlist

→ remove nth node from end

→ check if LL is a palindrome or not

Check if LL is a Palindrome

- ① find midNode ✓
- ② 2nd half Reverse ↩
- ③ check if 1st half (left) == 2nd half (right)

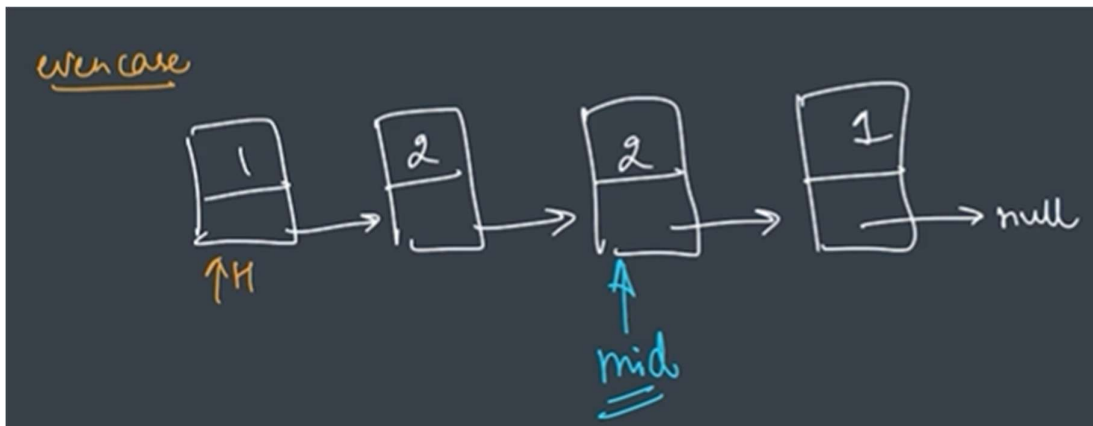
Step 1: find midNode

LL may be even or odd

Step2: after finding mid we do the reverse of 2nd half

Step3: check if 1st half = 2nd half

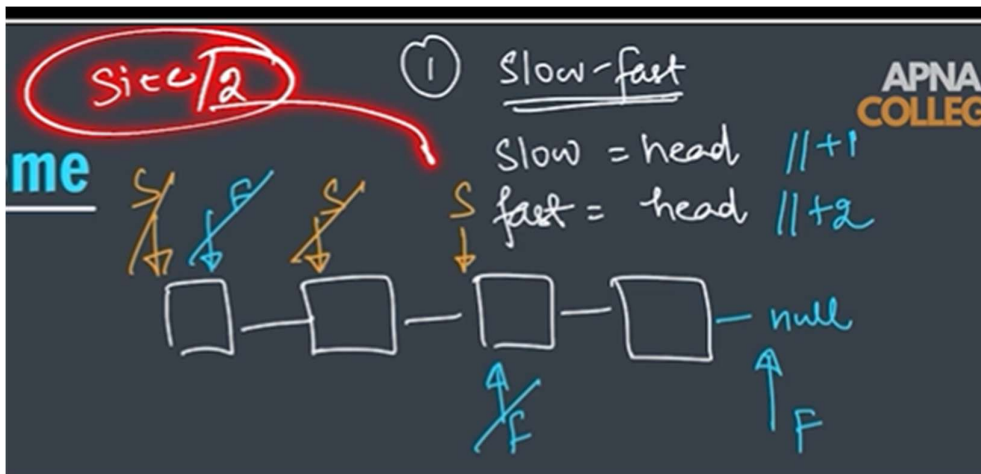
Even case:



→ slow-fast:

Slow=head; ++1

Fast=head ++2

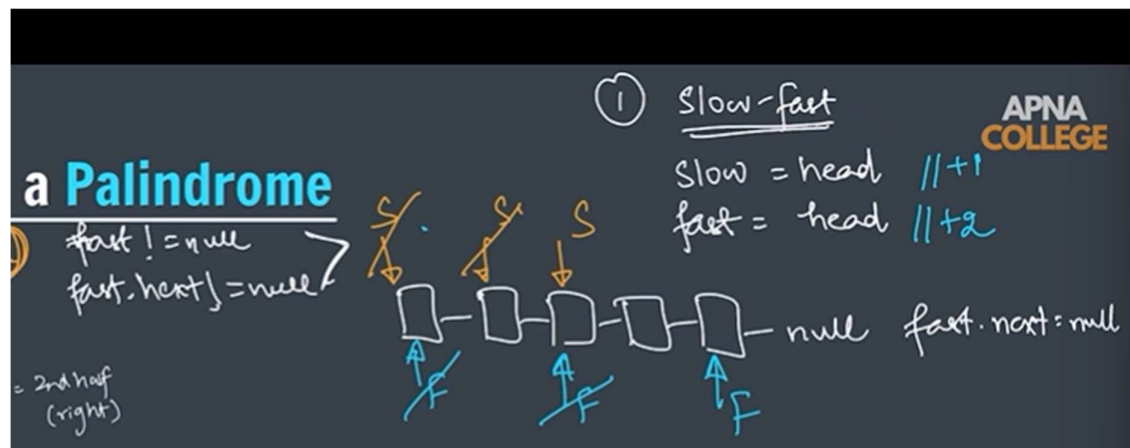


→ in even case we see that both the slow and fast initialized with the head but slow moves one at a time and fast moves two at a time

→ we should stop this process until $fast \neq null$ and $slow = size/2$;

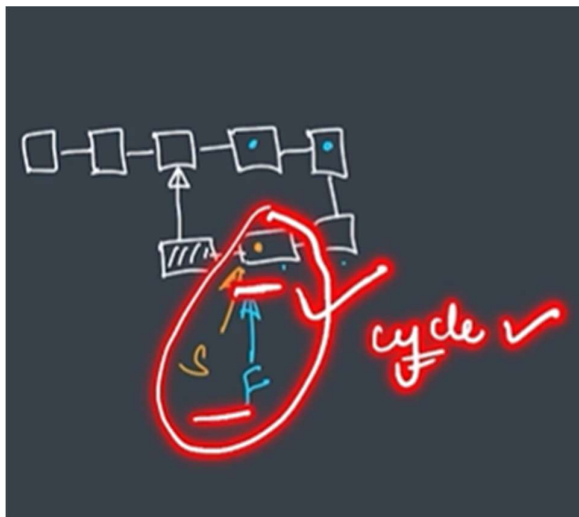
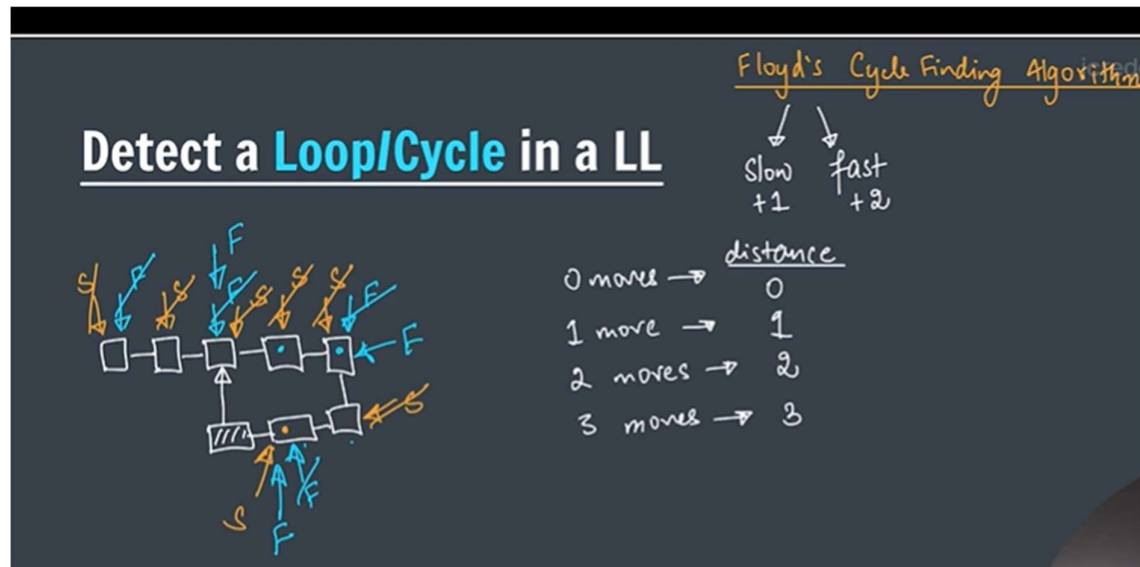
→ odd case:

In odd case fast should stop at the $fast.next = null$ and at point automatically $slow = mid$;



→ linked list part-2

→ Detect a loop/cycle in a ll



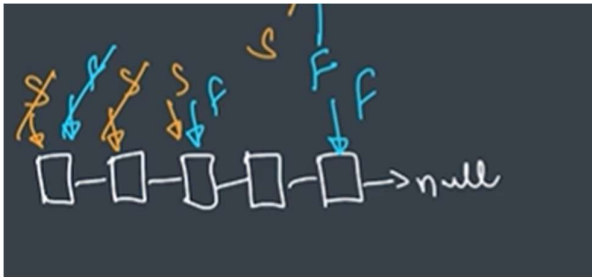
We initialize the slow and fast pointer as head

Slow=head;

Fast =head;

Then we start moving as slow=slow.next and fast=fast.next like they are moving in the ll then at a point when slow =fast at that point we can conclude that cycle is present.

→ in normal linked list they meet at starting but not meet at any point later .



Algorithm:

Slow=head;

Fast=head;

While(fast!=null&&f.next!=null)

{

Slow=slow.next;

Fast=fast.next.next;

If(slow==fast)

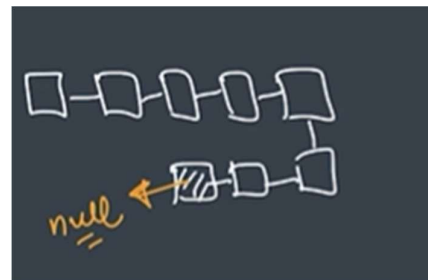
{

Return true;

}

Return false;

→ Remove a loop/cycle in a ll

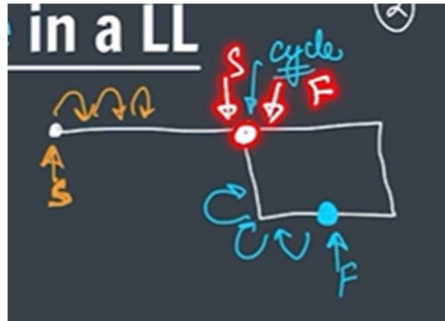


First we need to remove the link which is forming the cycle and make them equal to null.

→ to do that

Step1: find last node

Step2: lastnode.next=null;



→ algorithm:

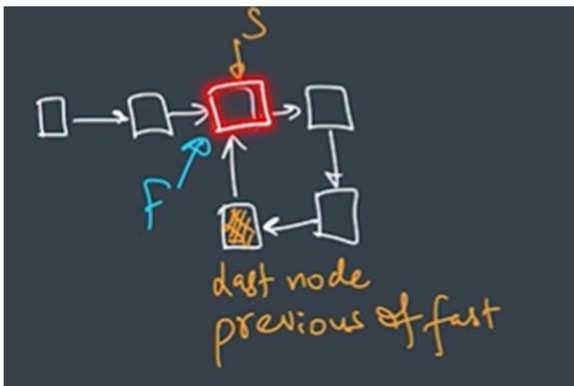
Step1: detect cycle

a) slow=head

b) slow->+1

fast->+1

prev=null



c) while(slow==fast)

{

Slow=slow.next;

Prev=fast

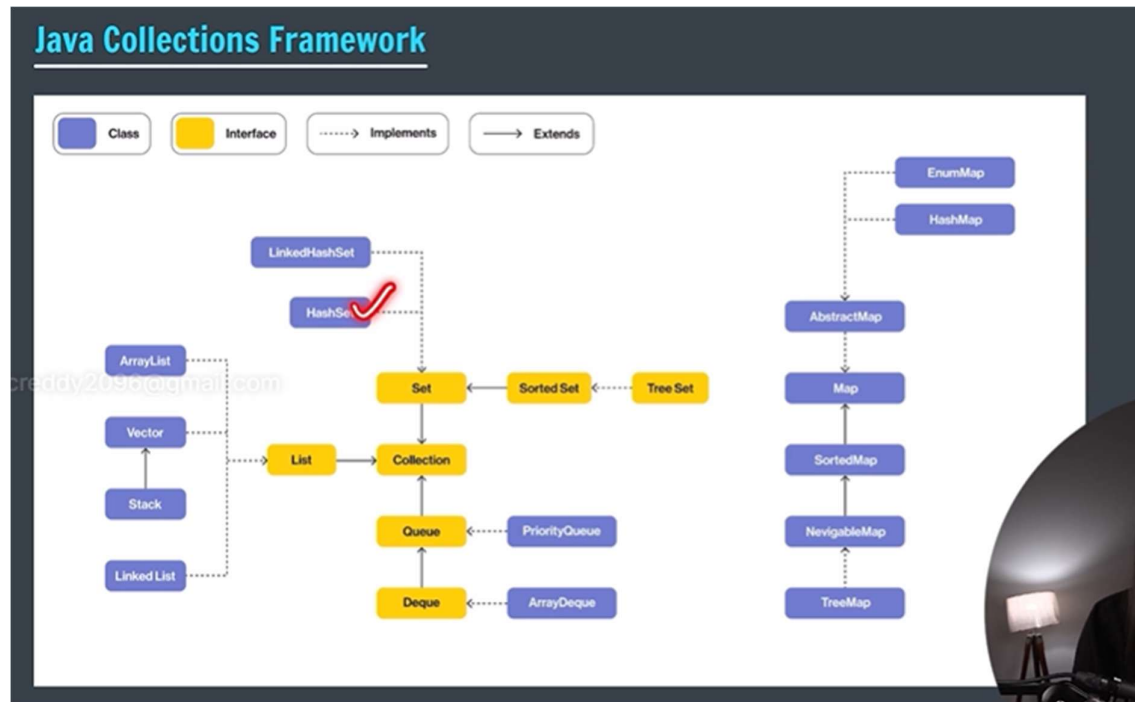
Fast=fast.next;

}

Prev.next=null //last node

→ Remove a loop/cycle in a ll (mathematical explanation)

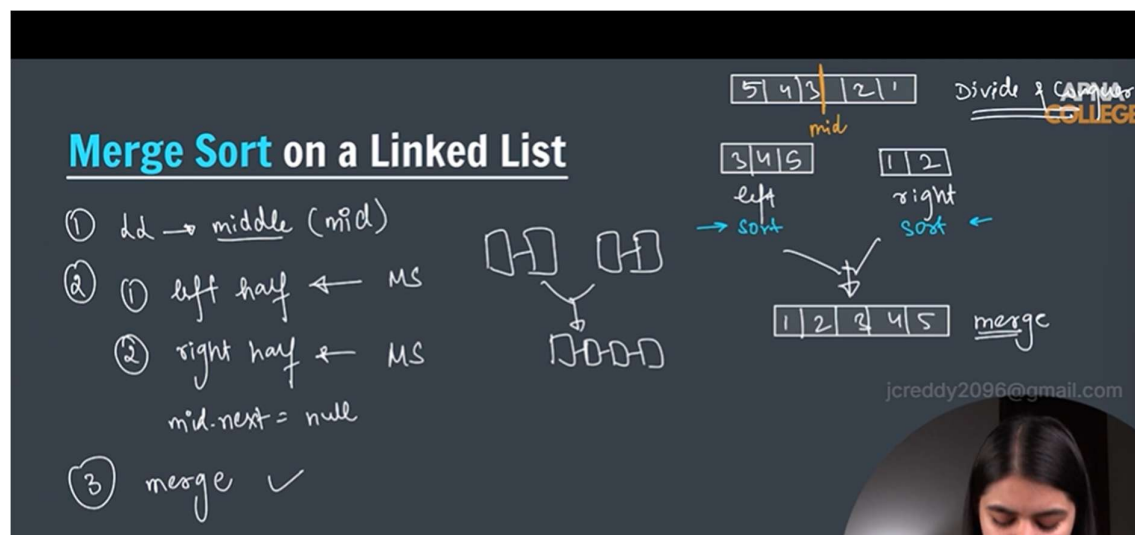
→ java collections framework:



→ ll in java collections framework

Mostly written in the code

→ Merge sort on a Linked List



→ step1: midNode

Slow=head;


```

Fast=head;

While(fast!=null&&fast.next!=null)

{

Slow=slow.next;

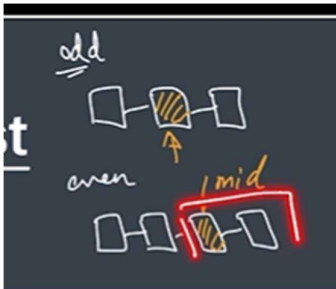
Fast=fast.next.next;

}

Return slow;

```

→ to find the middle node we use the above process but in this case we get for even no. of nodes we get the last node as next node first node



→ so, we initialize it as the fast=head.next

Merge Sort on a Linked List

① mid

```

slow = head
fast = head.next
while (fast != null && fast.next != null) {
    slow → +1
    fast → +2
}

```

slow → middle Node

LINKED LIST n=6

→ step2:

Righthhead=mid.next

Mid.next=null;

Mergesort(head)//lefthalf

Mergesort(righthalf)//right half

→ step3: Merge

Merge Sort on a Linked List

```
head1, head2
Node mergedLL = new Node(-1)
Node temp = mergedLL
while( h1 != null && h2 != null ) {
    h1 < h2
}
[ head1
  head2 ]
return mergedLL.next
```

→ zig-zag linked list

Zig-Zag Linked List

For a linked list of the form : L(1)→L(2)→L(3)→L(4) ... L(n-1)→L(n)
convert it into a **zig-zag** form i.e. : L(1)→L(n)→L(2)→L(n-1)→L(3)→L(n-2)...

1 → 2 → 3 → 4 → 5

1 → 5 → 2 → 4 → 3

→ approach:

even case:

1->2->3->4

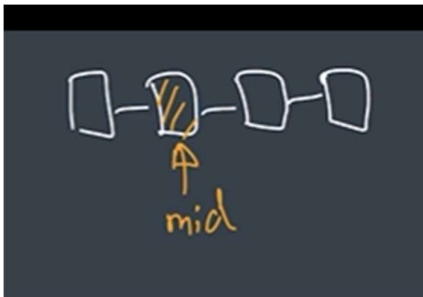
1->4->2->3

odd case:

1->2->3->4->5

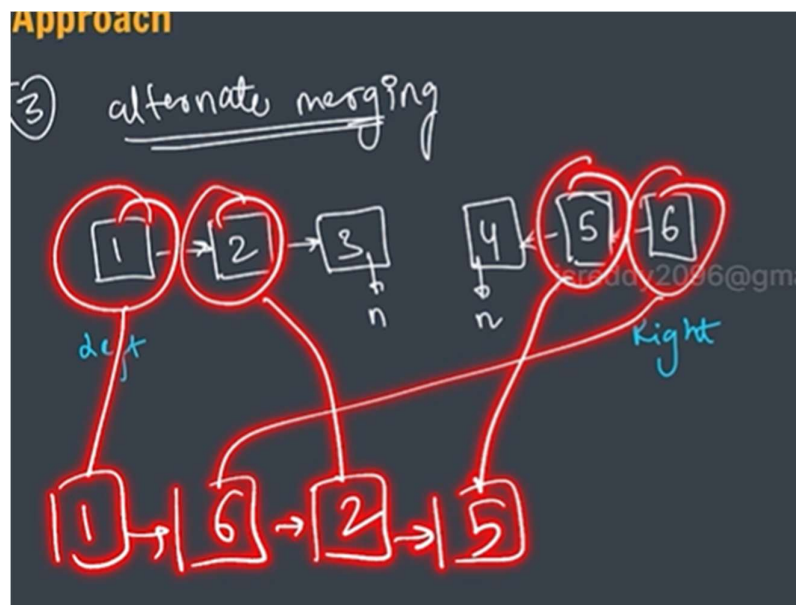
1->5->2->4->3

Step1: first find the midnode (1st half last node)



Step2: 2nd half reverse

Step3: alternate merging



Node lh=1st half head;

Node rh=2nd half head

Node nextL,nextR

③ alternate merging



Approach

③ alternate merging



Node LH = 1st half head

Node RH = 2nd half head

Node nextL, nextR

while (LH != null & RH != null) {

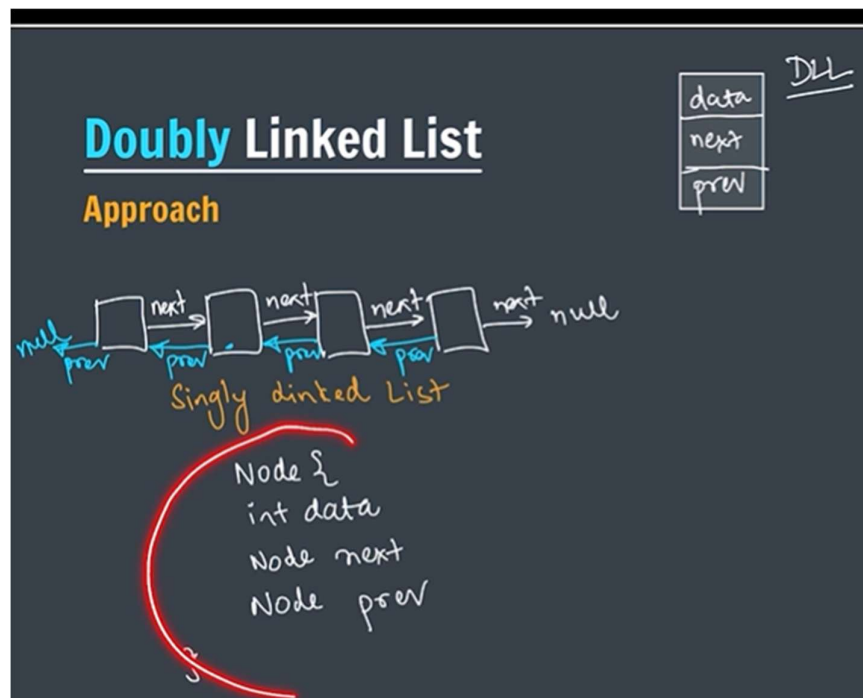
zigzag {

- nextL = LH.next
- LH.next = RH
- nextR = RH.next
- RH.next = nextL

update {

- RH = nextR
- LH = nextL

→ Doubly linked list



→ addFirst

Step1: create node

Step2: newnode.next=head;

Head.prev=newnode;

Head=newnode;

→ addLast

→ remove first

→ remove Last

→ circular linked list

